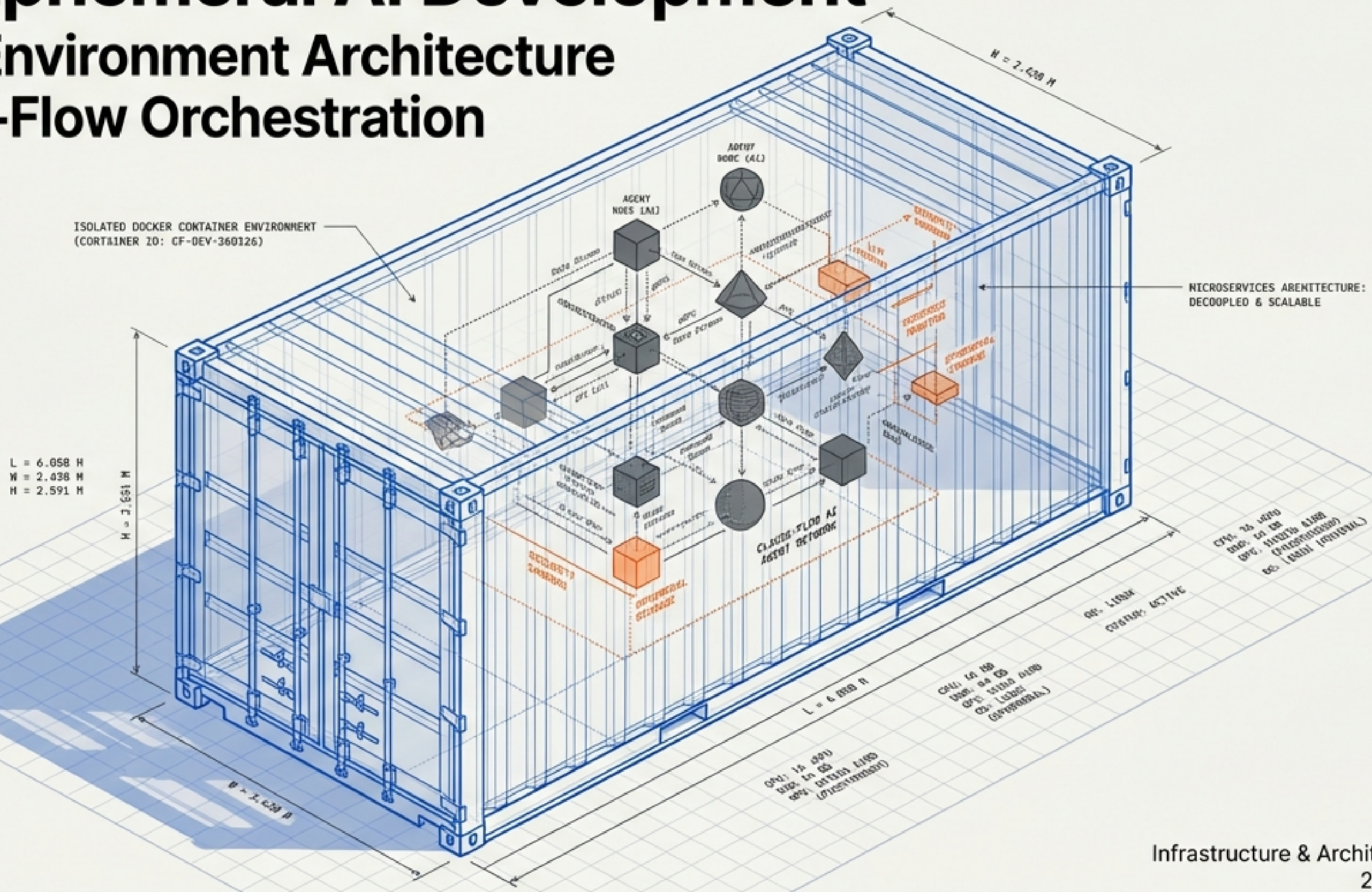
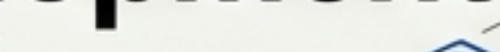


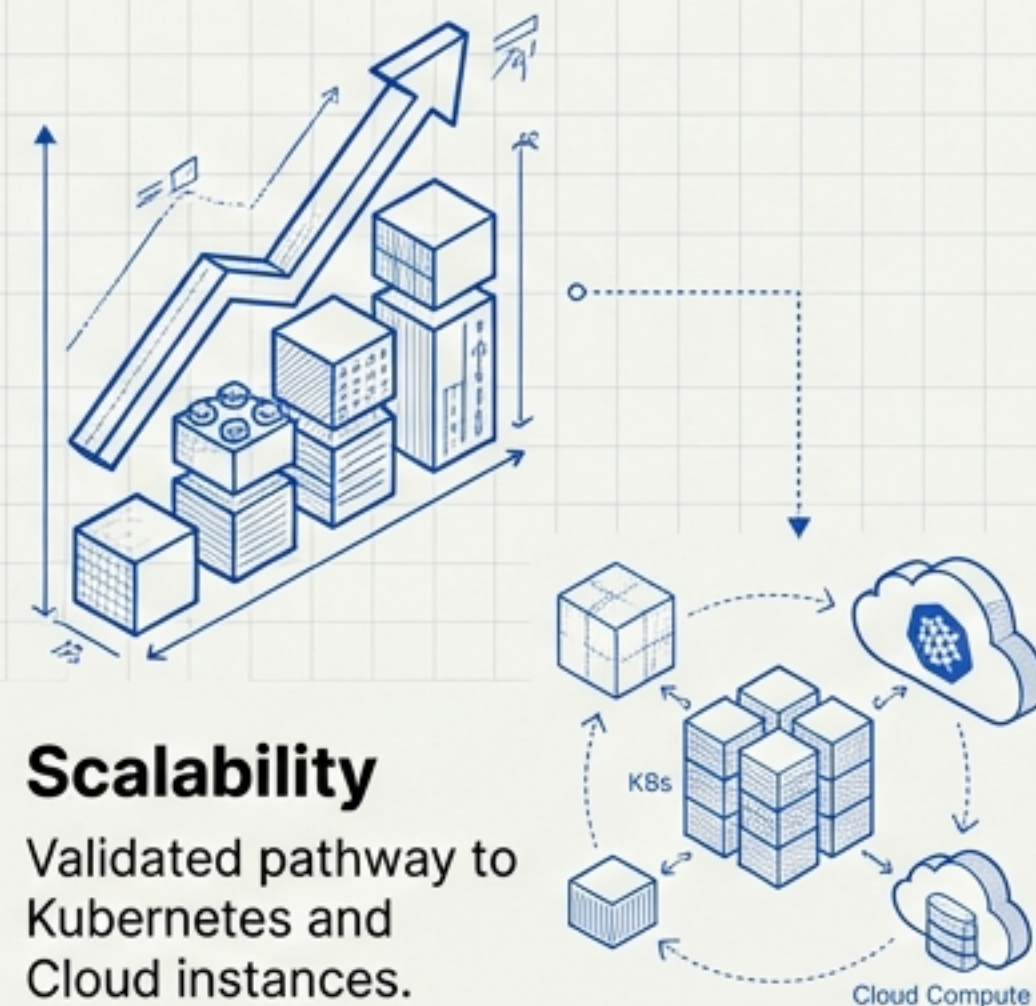
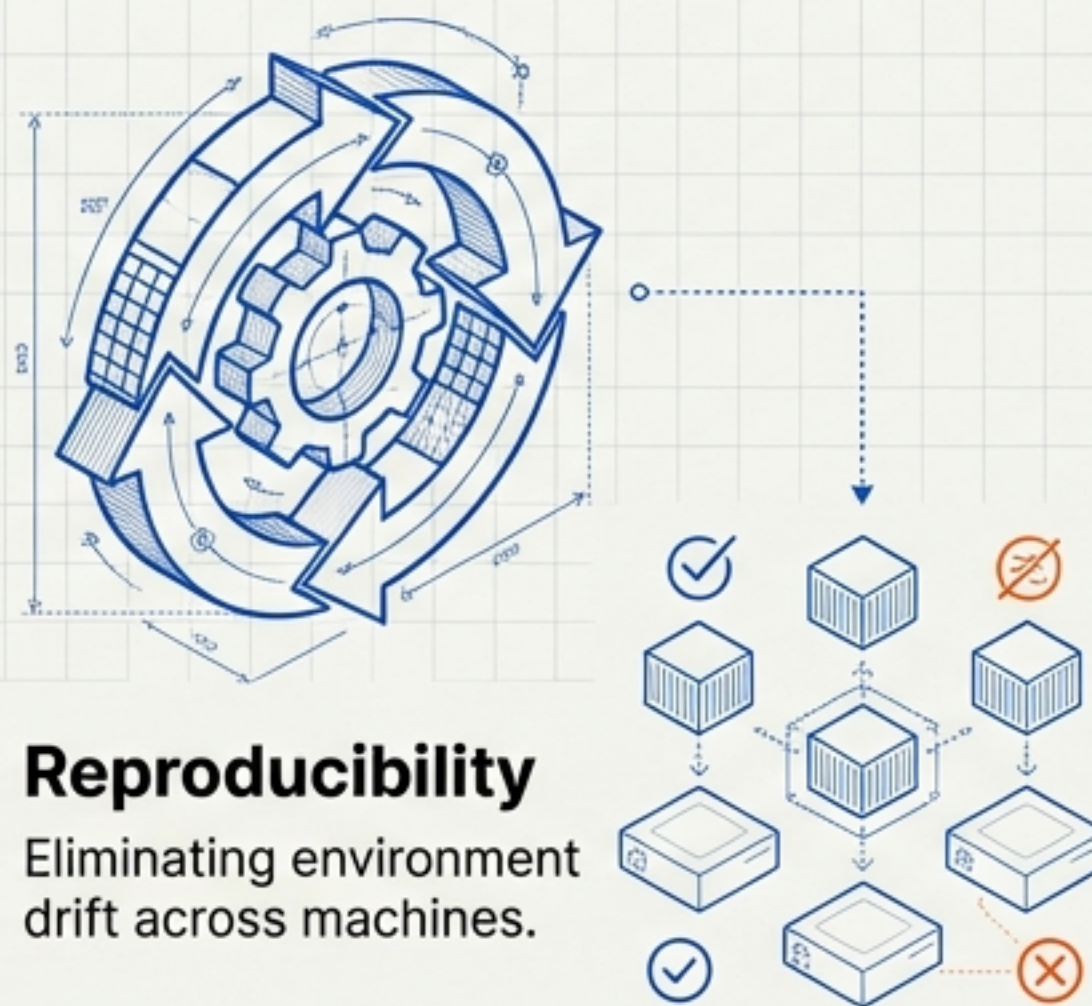
Safe & Ephemeral AI Development

A Docker Environment Architecture for Claude-Flow Orchestration



ARCHITECTURE OVERVIEW

This document outlines a purpose-built environment for safe Claude-Flow experimentation, serving as a proof-of-concept for production containerised deployments (K8s, Cloud Compute). It represents a transition from 'it works on my machine' to a standardised, ephemeral infrastructure.



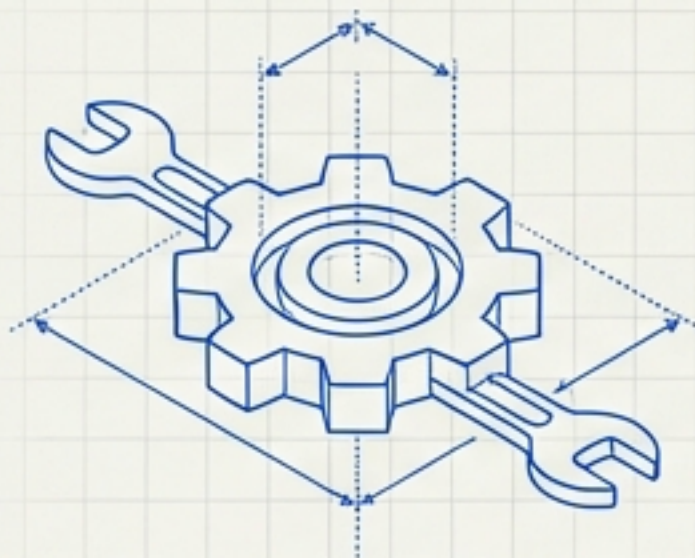
THE RISKS OF UNCONTAINED EXECUTION

Experimenting with AI agent orchestration systems like Claude-Flow on local hardware creates significant infrastructure liabilities.



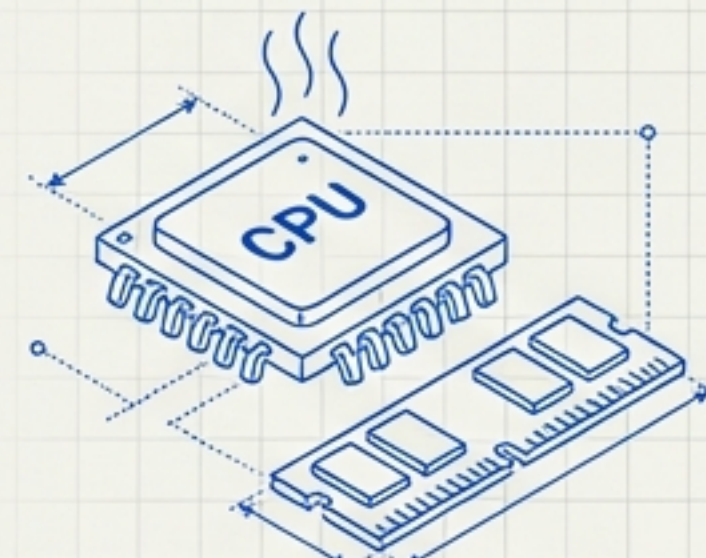
System Modification

Agents inherently attempt to write code and config. Risk of modifying system files or installing unwanted packages globally.



Resource Consumption

Parallel agents (e.g., 8-12 processes) can hog CPU/RAM, rendering the host machine unusable.



State Pollution

Experiments leave behind debris— orphaned files, databases, and cache layers that corrupt future runs.



Security Breach

Running autonomous agents with host system access introduces an undefined attack surface.



THE SOLUTION: A CONTAINERIZED SANDBOX

HOST MACHINE (PROTECTED)



Operating System Intact.

CONTAINER ENVIRONMENT (VOLATILE)



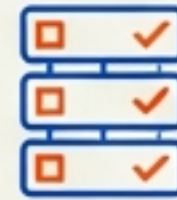
Active Experimentation.



Isolation: Nothing leaves the container unless explicitly mapped.



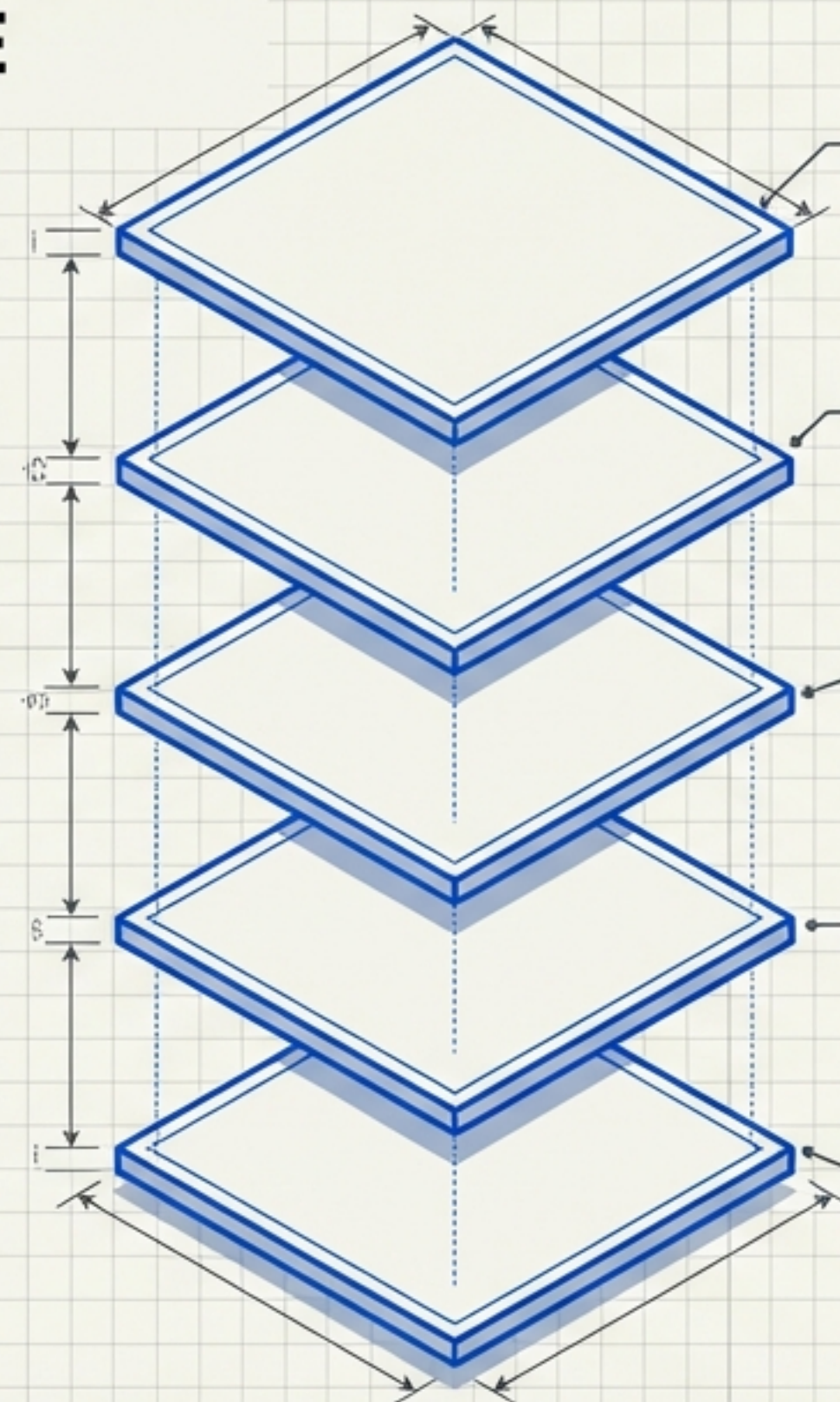
Ephemerality: The ability to wipe the slate clean instantly.



Standardisation: Every developer runs the exact same OS and dependency tree.

"A safe, isolated, and ephemeral manner."

BLUEPRINT ANALYSIS: THE DOCKERFILE



`npm install (local)`

Packages installed locally, not globally.
No root permissions needed.

User `claude`

Critical Security. Runs as non-root (UID 1000) to limit privilege escalation.

`rm -rf /var/lib/apt/lists/*`

Cleans package caches to minimize size.

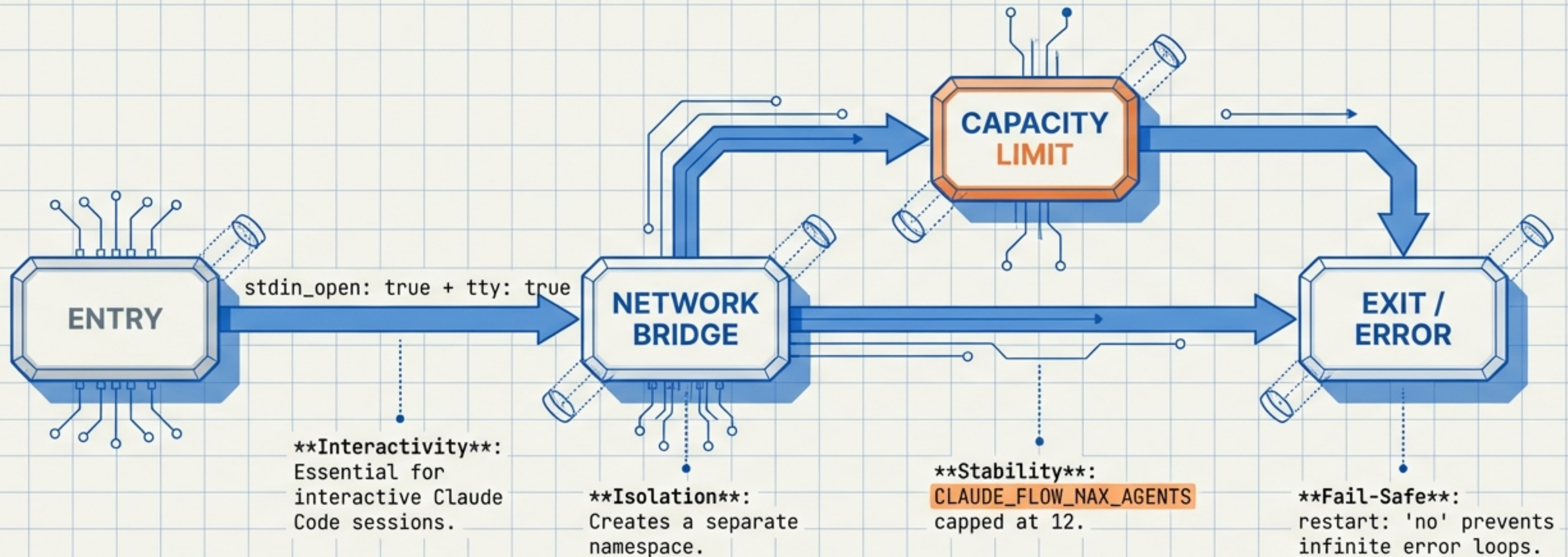
`--no-install-recommends`

Reduces attack surface and image bloat.

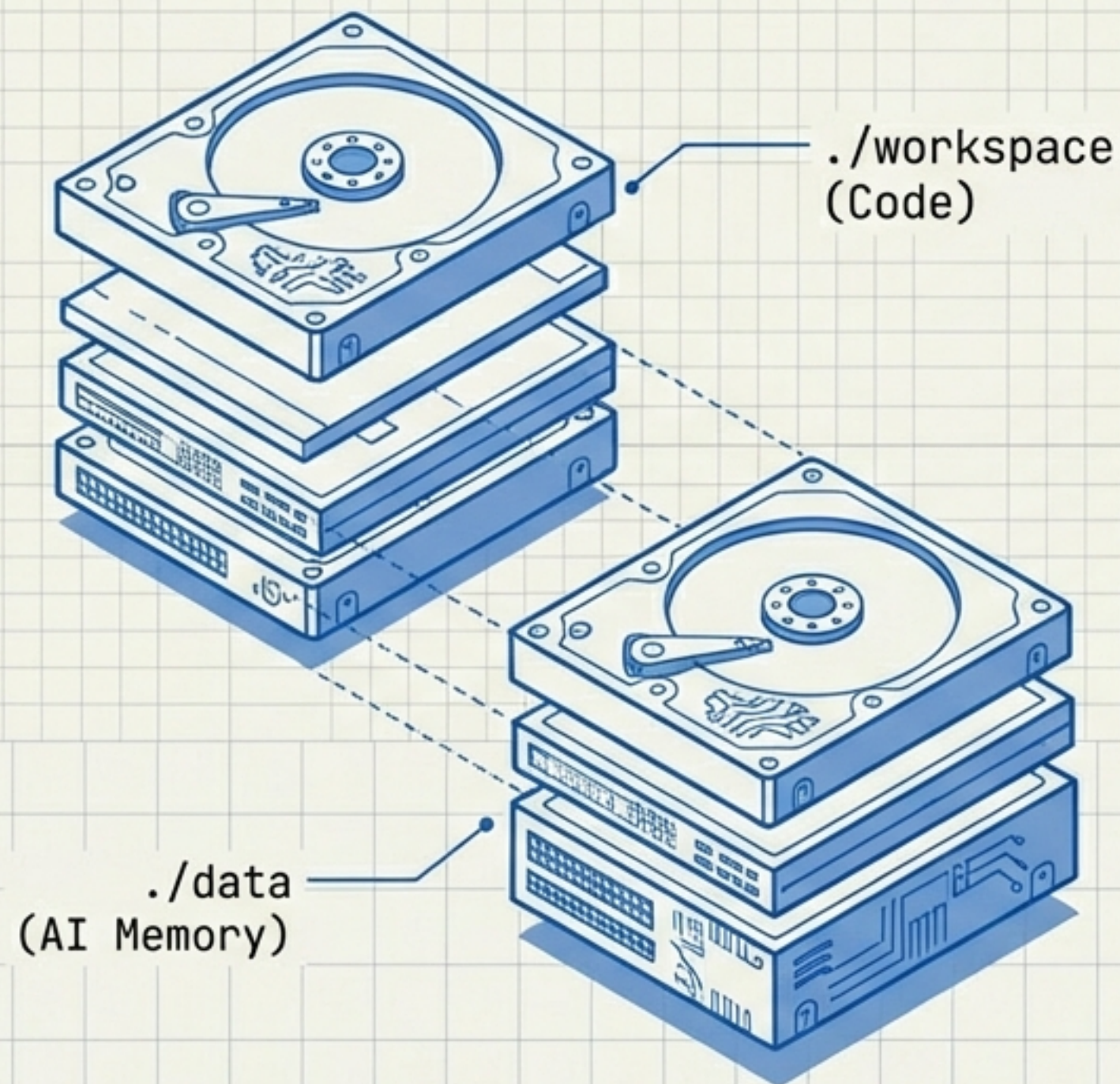
`node:20-slim`

Selected for LTS stability and minimal footprint (~200MB).

RUNTIME LOGIC & ORCHESTRATION



DATA STRATEGY: EPHEMERALITY VS. PERSISTENCE



Fresh Start

Action: Wipe
`./workspace & ./data`

Result: Complete
Reset



Keep Learnings

Action: Wipe
`./workspace` only

Result: Memory
preserved, code
reset



Reset AI

Action: Wipe `./data` only

Result: Code
preserved, AI slate
cleaned



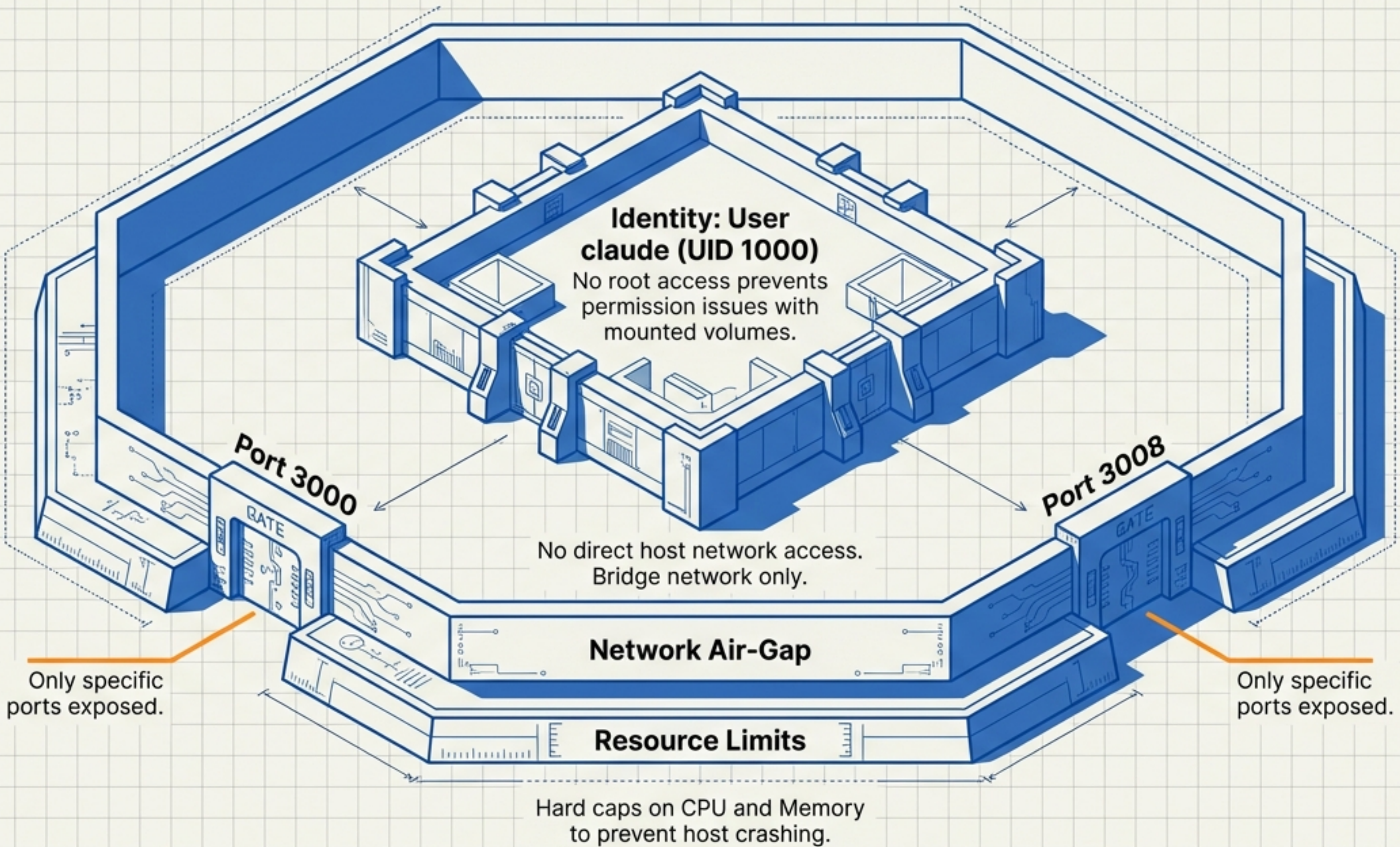
Full Persistence

Action: Wipe Nothing

Result: Resume
exactly where
left off

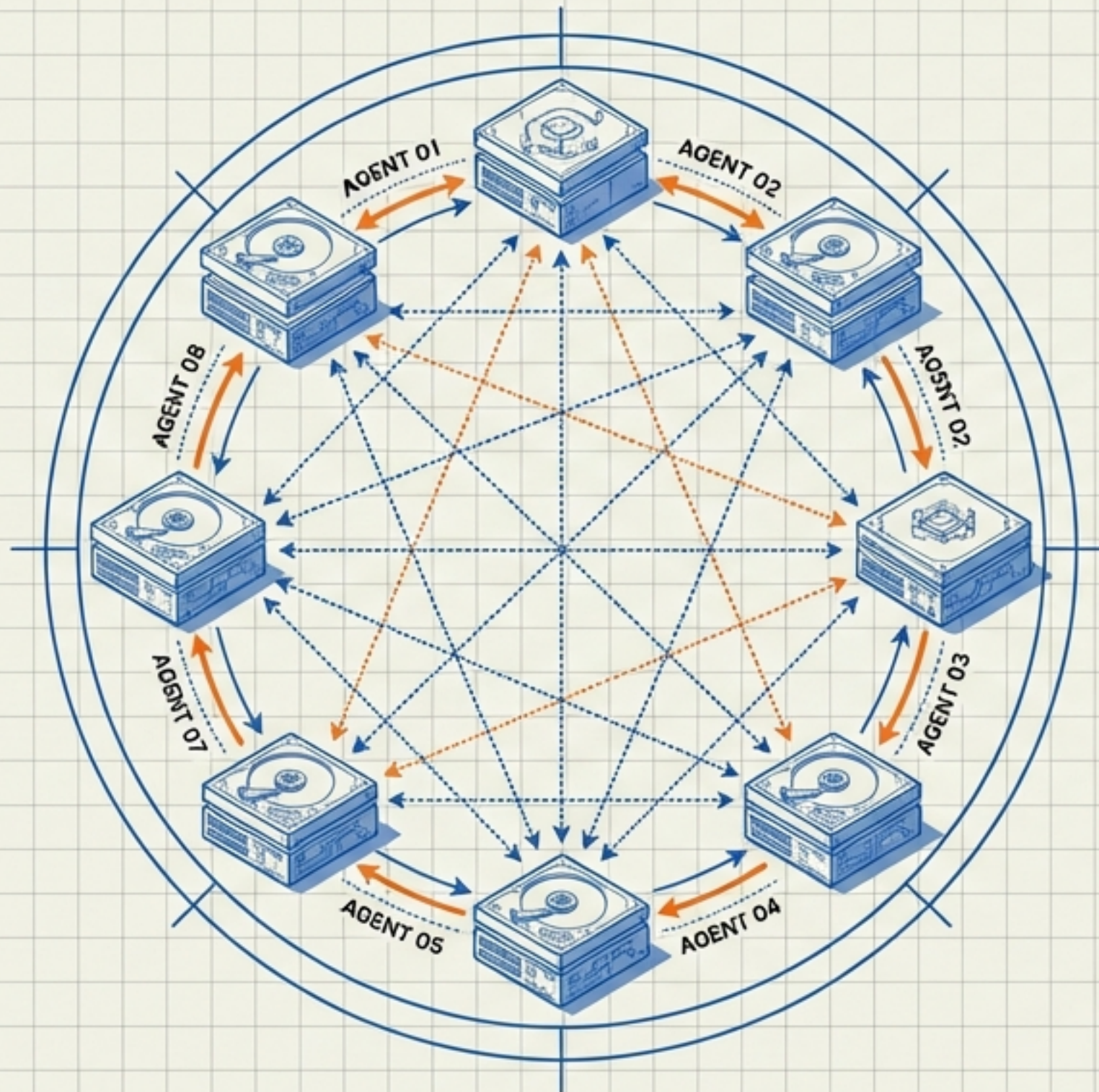


SECURITY ARCHITECTURE



VALIDATION: THE CRM PROJECT CASE STUDY

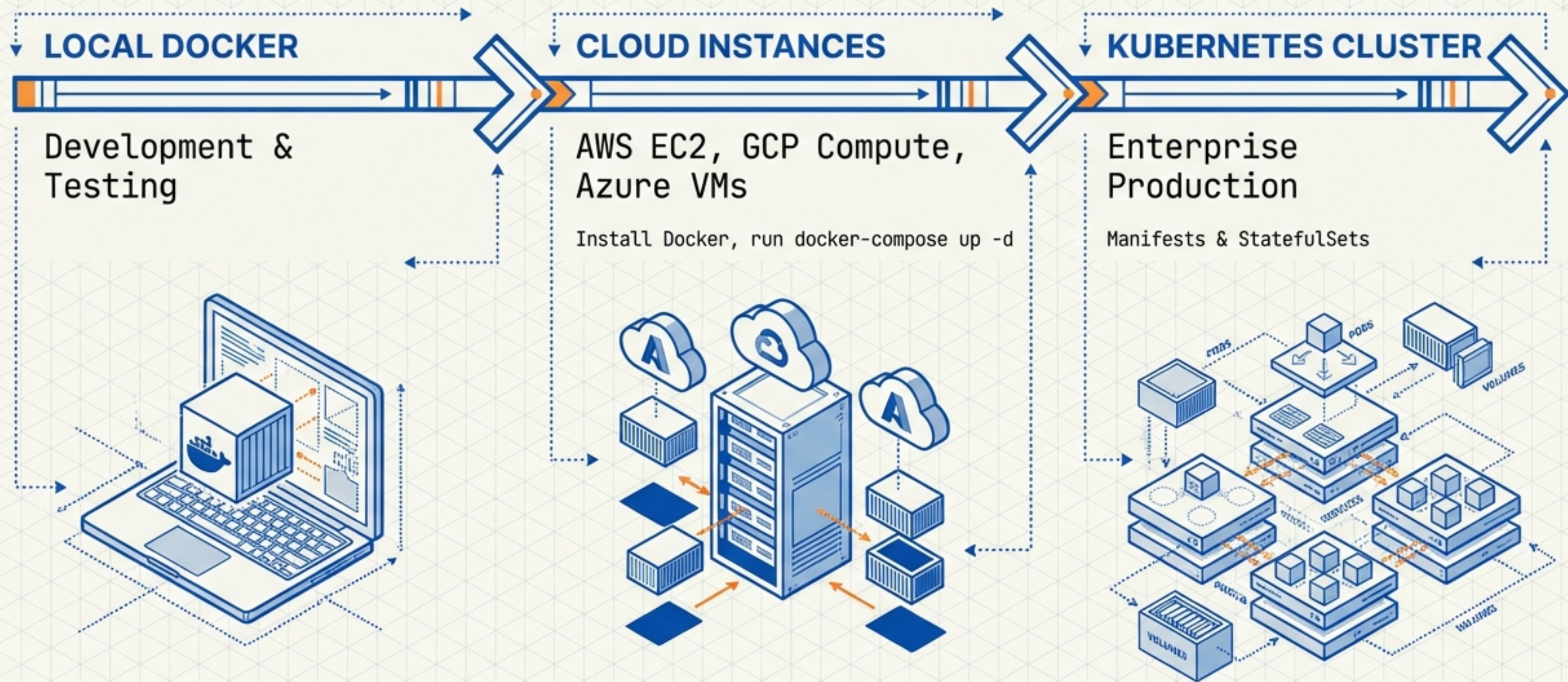
Proof of Concept running a complex Customer Relationship Management system.



VALIDATION METRICS

- 1. **ORCHESTRATION:** Successfully ran 8 agents in parallel execution.
- 2. **COMPLEXITY:** Proved that complex AI orchestration works in isolation.
- 3. **PRIVILEGE:** Confirmed non-root execution is viable without privilege escalation.
- 4. **PERSISTENCE:** Verified volume mounts allow for session resumes and memory retention.

SCALING TO PRODUCTION

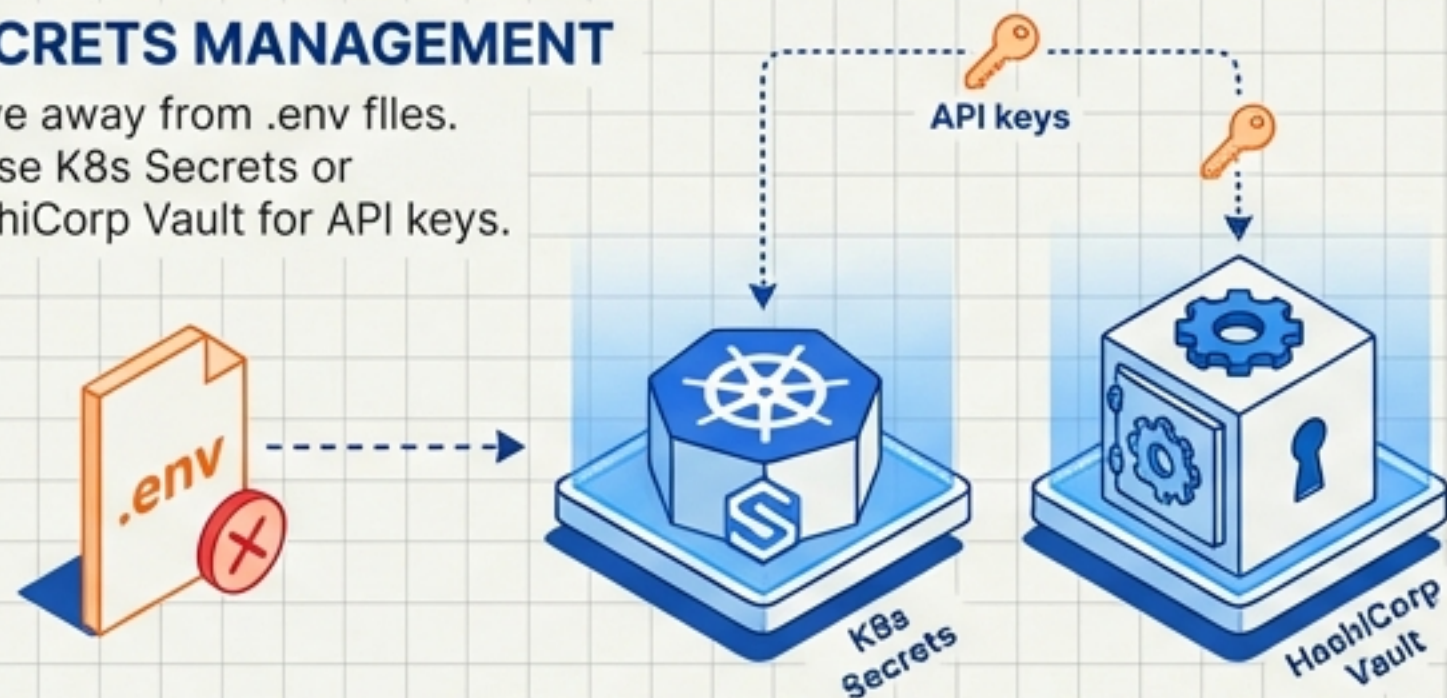


CI/CD Pipeline Integration: Validated for AI-assisted development workflows.

PRODUCTION BEST PRACTICES

SECRETS MANAGEMENT

Move away from .env files.
Utilise K8s Secrets or
HashiCorp Vault for API keys.



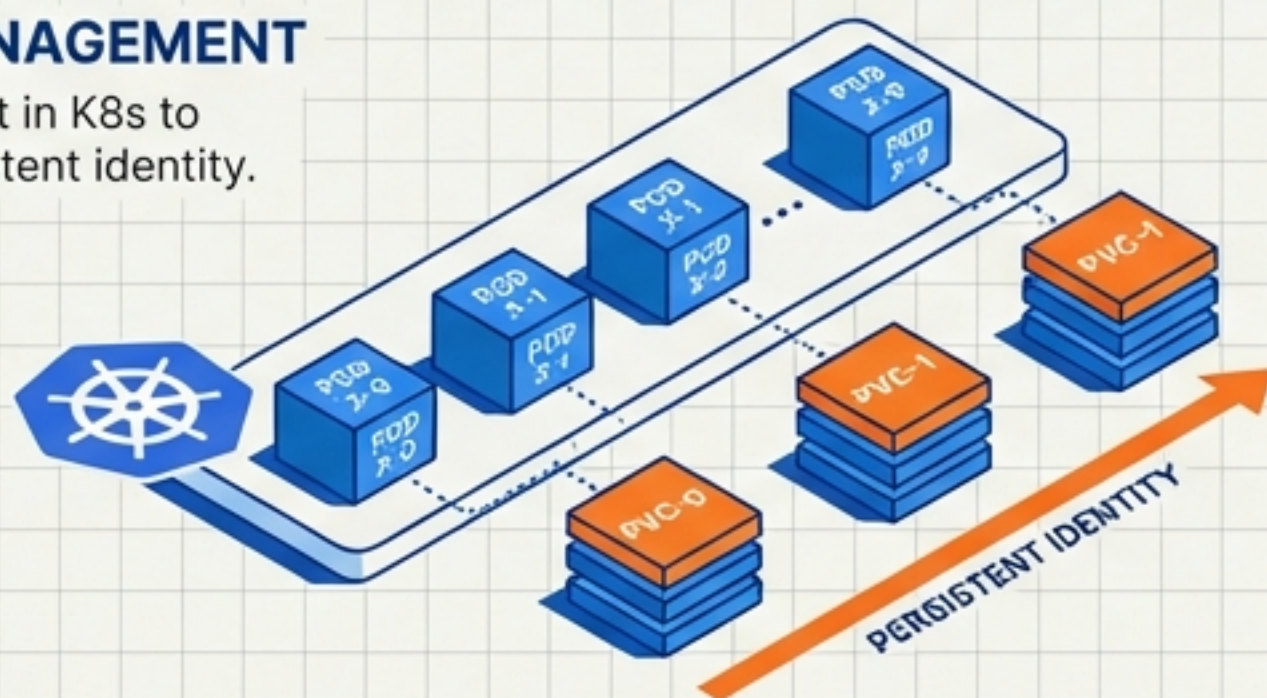
OBSERVABILITY

Implement structured logging
shipped to an aggregator and
Prometheus metrics for
agent health.



STATE MANAGEMENT

Use StatefulSet in K8s to
maintain persistent identity.



DATA INTEGRITY

Routine snapshots and
backups of ./data/
volumes.



CONFIRMED OPERATIONAL BENEFITS

OPERATIONAL CHECKLIST



Safe Experimentation:

- Risk-free sandboxing for volatile agents.



Total Reproducibility:

- Zero environment drift between developers.



Instant Ephemerality:

- Easy cleanup and reset of complex states.



Resource Management:

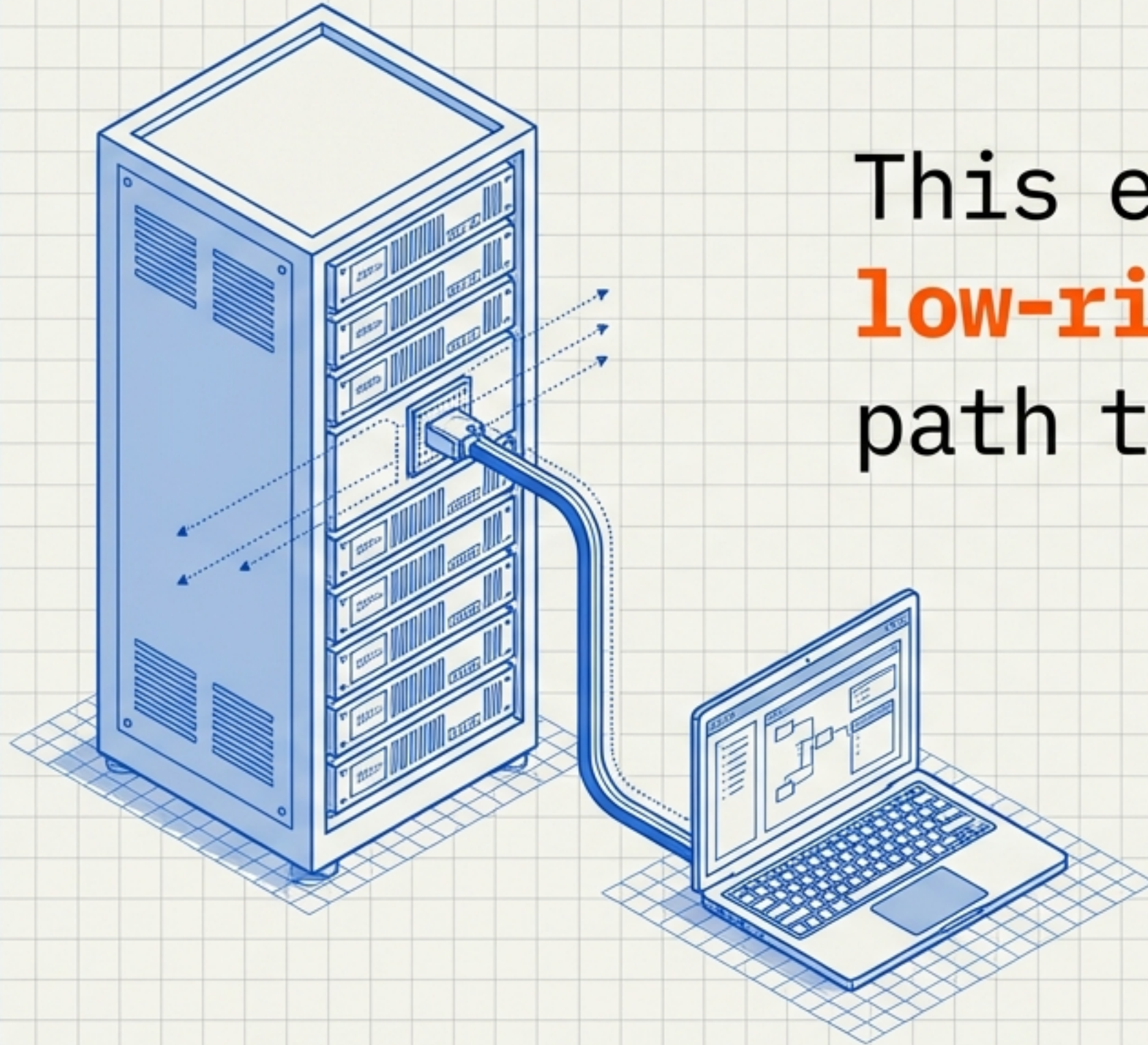
- Prevention of runaway costs and processes via docker stats and limits.

RISK vs. EFFICIENCY METRICS



THE PATH FORWARD

This environment provides a **low-risk, high-confidence** path to adopting AI agents.



- Complex AI orchestration works in containers.
- Non-root execution is viable.
- The setup is production-ready.

QUICK REFERENCE GUIDE

****Fresh Start (Total Reset)**

```
rm -rf ./workspace/* ./data/*
```

****Keep Learnings (Reset Code Only)**

```
rm -rf ./workspace/*
```

****Reset AI (Keep Code Only)**

```
rm -rf ./data/*
```

****Full Persistence*:** No action required; state is preserved automatically.

****Full Persistence*:** No action required; state is preserved automatically.

CREDITS & SOURCE

****Source Material****

Based on '006-docker-environment-design.md' –
Infrastructure & Architecture Analysis.

****Design****

Environment designed by Dinis Cruz.

****Context****

Part of the Claude-Flow Development Debrief Series.